

Lecture 5 - Time Series Analysis (AR and MA Models)

DS 4420: Machine Learning and Data Mining 2

Eric Gerber

`e.gerber@northeastern.edu`

Northeastern University, Boston, USA



Today

1 Time Series Data

2 $AR(p)$ Models

3 $MA(q)$ Models

4 Other Strategies

Today

1 Time Series Data

2 $AR(p)$ Models

3 $MA(q)$ Models

4 Other Strategies

Updates

- Homework 3 due **Friday, Feb. 13 by 11:59 pm**
 - Turn in by **Thursday, Feb. 12 at 11:59 pm** for 5% extra credit
 - Homework 4 will be posted then, due **Monday, Feb. 23 at 11:59 pm**
- Quiz 4 (on this material) will be **next Wednesday, Feb. 18**
- Project Guidelines released/presented this **Thursday**
 - [Team Choice Form](#) due **FOR ALL STUDENTS** by **Friday, Feb. 20**

Topics for Today:

- 1 Basic Time Series Models
 - Autoregressive (AR) Model
 - Moving Average (MA) Model

Push-Up Counter:

- Section 1: 10
- Section 4: 6

Today

1 Time Series Data

2 $AR(p)$ Models

3 $MA(q)$ Models

4 Other Strategies

Time Series Data

Often we break data into two types:

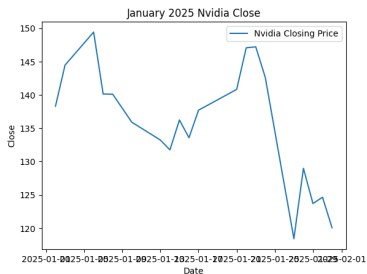
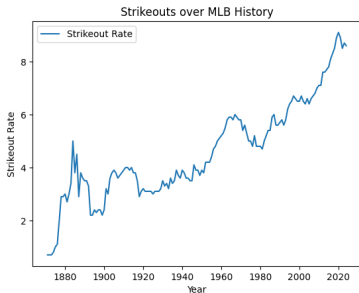
- **Cross sectional data:** where each observation is an independent sample collected at a single time point
- **Longitudinal/Time Series data:** where the data is made up of observations collected on one or more samples over time

The simplest form (which we will deal with today) is when you have a long sequence of values of a single variable over time. Some examples:

- A stock's value over time
- Temperature of a city over time
- An athlete's performance over time
- A student's grade over time
- etc.

Time Series Data

Time Series data is most easily and commonly visualized with a line plot, showing the value of the variable over time. We'll be making a lot of these:



The goal in Time Series Analysis is to use historical trends to **forecast** future values.

Reading Time Series Plots/Vocab

There are some common vocabulary terms that we can use to discuss Time Series data:

- **Level:** the average value in the series
- **Trend:** the long-term increase or decrease in the data
 - What were the trends in the plots on the previous slide?
- **Seasonality:** repeating patterns based on a specific time period
 - Sometimes time series data does not have strong seasonality, though many (things like temperature) do
- **Noise:** the random variation in the series

Time Series Data

There are many ways to analyze Time Series data. Mixed effects models and Bayesian hierarchical models (which we will discuss later in the semester) are some of the more complicated ways. There are also **A LOT** of more traditional regression type techniques, of which we'll at least cover the first two:

- Autoregressive (AR) Models
- Moving Average (MA) Models
- ARMA (combines the first two)
- ARIMA/SARIMA (combines the first two and allows for non-stationary processes)
- LSTM (which is a neural network approach)
- Many, many, more...

Assumptions of Time Series Data

Before continuing with analysis, there are a few assumptions we make about the structure of the data:

- Data should be in **chronological order** (earlier dates first, more recent dates last)
- Timestamps should be **equidistant** (the time between each observation should be the same)
 - If this is not the case, you should use **resampling**
- Ideally **no missing data**. If there is, we must address it. There are multiple ways to do this:
 - Drop and downsample (reasonable if you have a lot of data)
 - Fill with zero (suboptimal)
 - Fill with interpolated values (reasonable if you have limited data, but requires knowledge of the neighboring value)

Resampling

There are two types of resampling which can either/both (a) help deal with missing values or/and (b) allow the models to fit the data better:

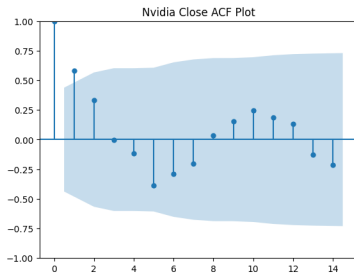
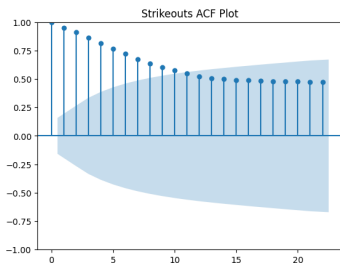
- **Downsampling** is when you decrease the frequency of the samples (i.e. from days to weeks) which averages the values within the new bins. This is straightforward and can deal with the missing values or filter some of the noise to allow the model to capture larger trends in the data.
- **Upsampling** is more complicated, when you increase the frequency of the samples (i.e. from days to hours), but requires more complex strategies. We won't touch it in class, but you can explore it on your own if you use Time Series in your projects.

More Assumptions of Time Series Data

- For the basic models we discuss, we assume the series is (weakly) **stationary**: that the errors/shocks behave like white noise. We will talk about a simple way to check for this, but what this means is:
 - the time series has constant mean
 - the time series has constant variance
 - the correlation/covariance between values of the time series depends only on how far apart they are (**lag** - see below), not *when* they occur.
- Values at certain time points are **correlated** with recent values up to some **lag**.
 - We can examine the **autocorrelation function** and **partial autocorrelation function** plots to give us some idea of how many lags might be useful.
 - For example, a **Lag of 1** would mean that we are going to use only y_{t-1} to predict y_t (or, we assume only the previous year contains important information (is correlated) with the next year).
- Cross validation takes place by using the **earlier** data as training data and the **later** data as test (no longer randomly splitting!)

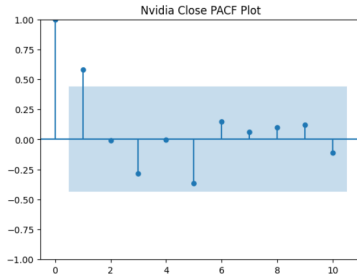
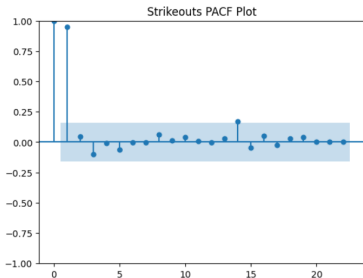
Autocorrelation Function (ACF) Plot

Here are some examples of ACF plots for the Strikeouts and Nvidia time series data from Slide 11:



Partial Autocorrelation Function (PACF) Plot

Here are some examples of PACF plots for the Strikeouts and Nvidia time series data from Slide 11:



- We'll talk about how to interpret these on the next slide, and how to prepare our data for modeling, using **time_series.ipynb** and **time_series.R**, before moving on to discussing the first model.

ACF vs. PACF

Autocorrelation	Partial Autocorrelation
Identifies likely order of moving average (MA) process at q , where the ACF cuts off	Identifies likely order of autoregressive (AR) process at p , where PACF cuts off
Represents overall correlation structure of series	Highlights direct relationships between observations at specific lags
Tails off (geometric decay) suggests AR process	Tails off (geometric decay) suggests MA process

- If both ACF and PACF plots tail off (geometric decay), it implies the process has both AR and MA components. If both plots cut off quickly, it implies simple models may not be appropriate, or the data are too noisy.

Today

1 Time Series Data

2 $AR(p)$ **Models**

3 $MA(q)$ **Models**

4 Other Strategies

Autoregressive Time Series Models

The simplest type of Time Series model is the Autoregressive (AR) Model, often called the $AR(p)$ model, where p is the lag used. The basic formulation is, if y_t is the value of the time series at time t :

$$y_t = \sum_{i=1}^p w_i y_{t-i} + \varepsilon_t$$

Another way to think about it: the $AR(1)$ model is simply a simple linear regression model predicting y_t with y_{t-1} :

$$\hat{y}_t = w y_{t-1} \quad \text{or} \quad \hat{y}_t = w_0 + w_1 y_{t-1}$$

- You can fit AR models with or without an intercept. Including an intercept allows a nonzero mean level (also known as **drift**).

$AR(p)$ Models

Since the **lagged values** are serving as our inputs for the regression, we often denote them X . For example, assume you have the following time series data, where say the y are the hours you spent sleeping each night over a school week (M-F). An $AR(1)$ model would have the x - y pairs:

Night	y		$x = y_{t-1}$	y
1	8.0		8.0	7.4
2	7.4		7.4	7.2
3	7.2		7.2	7.6
4	7.6		7.6	8.2
5	8.2	$\rightarrow$		

- **NOTE:** depending on the lag, you will lose a certain number of observations, since you need to use their previous data to predict the y_t value.
- For example, an $AR(2)$ X matrix and y vector for the above (ignoring intercept):

$$X = \begin{bmatrix} 7.4 & 8.0 \\ 7.2 & 7.4 \\ 7.6 & 7.2 \end{bmatrix} \quad \text{and} \quad y = \begin{bmatrix} 7.2 \\ 7.6 \\ 8.2 \end{bmatrix}$$

AR(p) Models

Once you define the X matrix given the choice of p (lag), you simply fit the model as you would any OLS:

$$w = (X^T X)^{-1} X^T y$$

Easy enough? Check out the **time_series** python and R files to see it done with some real data first by hand, then using prebuilt packages.

Predictions Using $AR(p)$

As seen in the python and R examples, predictions on the test data (to assess performance) are not done exactly the same way as predictions in regular regression.

Since these are time series data, we cannot assume we know any of the "future" values. Suppose we have the $AR(1)$ model and assume your training data goes from $y_1, \dots, y_t$:

- Predict $\hat{y}_{t+1} = wy_t$
- Predict $\hat{y}_{t+2} = w\hat{y}_{t+1}$
- ...

So, predictions of future values depend on the last observed value, and then the patterns/trends captured by the weights. Fitting a model with more lagged data (larger p) will therefore generally do a better job of capturing trends.

Evaluating $AR(p)$ Models

If you are unsure about which p to use, there are a few diagnostics you can use to help:

- Make sure that the **roots (in modulus)** of the model are all > 1 ; this ensures that the assumption that the series is **stationary** is reasonably met
 - Sometimes, if the roots in modulus are very close to 1, we'll hand wave and say it's fine
 - But if one model is stationary and another isn't, you should choose the stationary one!
 - When dealing with multiple lags, you can check the roots in modulus with `abs(ar_results.roots)`
- Compare plots of the true test data with the predictions under the model
- There are many different **Information Criterion**, some of which are provided when you fit the model with prebuilt packages. Generally, lower values represent better fits.
- You can compare the mean absolute and mean squared (or root mean squared) errors (MAE , $MSE/RMSE$)
- You can evaluate residual plots (just like in any regression) to see if the residuals are still autocorrelated (which suggests the model hasn't captured all time dependence; you might increase p or move to ARMA/ARIMA) and roughly normal (if they are not, you might try some data transformations)

Benefits/Drawbacks of $AR(p)$ Models

Benefits:

- **Simple and Interpretable:** if you have a basic understanding of linear regression, you can understand $AR(p)$ models!
- Efficient and useful for short-term dependencies with low noise:
 - If the time series has decently strong patterns and the data are stationary, the $AR(p)$ model does a great job and is way more efficient than more complex models.

Drawbacks:

- $AR(p)$ models are designed for **stationary** series; but more complex models allow for a stochastic trend (also called an **Integration filter**: the **I** in **ARIMA** and **SARIMA**).
- $AR(p)$ models are not great at capturing long-term patterns, since increasing p is something of a non-scalable way of increasing the complexity of the model.
 - Related, choice of p is tricky. Too low and you underfit, too high and you overfit (this is why cross validation is so important!)
 - Also related, you wouldn't really use them for long-term forecasting (something like **ARIMA**, **SARIMA**, or **LSTM** would do better).

Practice

Given the **air_traffic.csv** file on Canvas, take some time to practice implementing $AR(p)$ models both by hand and using either python or R. We'll take an informal survey when you're done to figure out which p we should use (and Dr. Gerber will show you what p he chose).

Today

1 Time Series Data

2 $AR(p)$ Models

3 $MA(q)$ Models

4 Other Strategies

Moving Average Time Series Models

Another simple model, at least conceptually, is the Moving Average (MA) Model, often called the $MA(q)$ model, where q is the lag used (more commonly referred to the **order** of the model). The basic formulation is, if y_t is the value of the time series at time t :

$$y_t = \mu_y + \sum_{i=1}^q \theta_i \varepsilon_{t-i} + \varepsilon_t$$

There are a few differences to the $AR(p)$ model here:

- μ is the level (or the mean) of the time series
- Notice we are not using the previous values of the time series directly, but rather **white noise terms**, $\varepsilon_t, \dots, \varepsilon_{t-q}$ associated with the time series based on the chosen lag
 - For example, this means if we overestimate the value at time $t-1$, the model adjusts by predicting something slightly lower at time t .
- The θ_i are the coefficients of the model, also called the **moving average constants**

The $MA(1)$ Model as an example

If we consider the $MA(1)$ model, we note that the present value of the time series will only depend on a single past error term, and thus will have what we call a "short memory". Rearranging the terms can help us understand:

$$y_t = \mu + \varepsilon_t + \theta_1 \varepsilon_{t-1}$$

Larger values of q give the $MA(q)$ model a longer memory and dependence on the past values. However, notice also that this means:

- $MA(q)$ is more complicated than $AR(p)$, because we cannot observe the lagged error terms (ε_{t-q}) directly; we will need to use an **iterative** fitting procedure in place of OLS.

Difference between $AR(p)$ and $MA(q)$

Use the $AR(1)$ and $MA(1)$ models to compare the two:

$$y_t = \mu + \varepsilon_t + \theta_1 \varepsilon_{t-1} \quad \text{and} \quad y_t = \varepsilon_t + w y_{t-1}$$

- Notice that the past noise term ε_{t-1} affects the MA model's present value y_t directly, but it only has an indirect affect on the AR model's present value (since $y_{t-1} = \varepsilon_{t-1} + w y_{t-2}$)
- This difference results in considering:
 - The $AR(p)$ model is an **infinite** impulse model. Each value is affected by **ALL** previous noise terms, though to a lesser extent the further back you go
 - The $MA(q)$ model is a **finite** impulse model. Each value is affected by only the current and previous q noise terms.

Assumptions of $MA(q)$ Models

Unlike $AR(p)$ models, $MA(q)$ models are stationary by construction (assuming noise has constant variance).

- The roots of the $MA(q)$ model can still be checked, but they tell us if the process is **invertible** (ensures uniqueness of the model), and software will usually enforce this for us.

When the observed data is nonstationary, these models (like $AR(p)$) can perform poorly. A common remedy is to apply **differencing** to the data:

- There are a few different types of differencing, but the simplest form is called **first order differencing**:

$$y_t^* = y_t - y_{t-1}$$

- Using y^* (the "differenced" time series values) can help remove trend (and make the mean more stable), which often makes AR/MA type structure easier to model. (Predictions are made on the differenced scale and then converted back to the original scale.)
- Seasonal structure may require *seasonal* differencing or components (i.e. a *SARIMA* model; more on that later)

Assumptions $MA(q)$ Models

While we had **white noise** in the $AR(p)$ model (via the ε_t terms), it becomes more important to the discussion of $MA(q)$ models because they are the "inputs" to the model. Recall that the $MA(q)$ model is a **finite** impulse model. Each value is affected by only the current and previous q noise terms (for example, the $MA(1)$ model):

$$y_t = \mu + \varepsilon_t + \theta_1 \varepsilon_{t-1}$$

- Past errors (more commonly called **shocks**) drive the series: ($\hat{\varepsilon}_{t-1} = y_{t-1} - \hat{y}_{t-1}$; where $\hat{y}_{t-1}$ is the predicted value at time $t-1$), as such the modeling is not as straightforward as OLS.
- As such, we won't bother fitting the $MA(q)$ models by hand, and will instead use pre-built packages.
 - In practice, the shocks are estimated from residuals during fitting, which is why estimation is iterative

Drawbacks of $MA(q)$ Models

Depending on the data, $MA(q)$ models may forecast better or worse than $AR(p)$; they capture different types of dependence. **Certain data is more appropriate for $AR(p)$ and certain data is more appropriate for $MA(q)$** (and, you can also make $ARMA(p, q)$ or $ARIMA(p, d, q)$ models which combine the two):

- $AR(p)$ data assumes that it is the **value** of the past lags that are informative to the current value
- $MA(q)$ data assumes that it is the **error/white noise** of the past lags that are informative to the current value
- Either or both can be true (hence the need for separate models and combined models)
- To help tell, you can look at both autocorrelation (ACF) plots and **partial autocorrelation (PACF) plots**
 - If the ACF has a drastic cut-off (does not decay slowly), the data are more likely to follow an $MA(q)$ process
 - Use the ACF to give a rough idea of q (what's the last significant lag?)
 - If the PACF has a drastic cut-off (does not decay slowly), the data are more likely to follow an $AR(p)$ process
 - Use the PACF to give a rough idea of p (what's the last significant lag?)

Practice

Given the **nvidia_jan25.csv** and **air_traffic.csv** files on Canvas, use the ACF and PACF plots to determine if the $AR(p)$, $MA(q)$, or perhaps both/neither model(s) are appropriate for analyzing those data.

Today

1 Time Series Data

2 $AR(p)$ Models

3 $MA(q)$ Models

4 Other Strategies

Other Strategies

While $AR(p)$ and $MA(q)$ models serve as a starting point, and can work well with simple problems, they are often a bit *too* simple by themselves. There are many ways to extend/improve upon them:

- We've already talked about $ARMA(p, q)$ which combine the two. There is also $ARIMA(p, d, q)$ which adds differencing to model nonstationary series, or $SARIMA(p, d, q, s)$ which adds explicit seasonal components.
- You can also add additional input features with $ARX, MAX, ARMAX, ARIMAX, SARIMAX$ models.
- $LSTM$ are commonly used neural networks for time series data, but you can also use MLP or CNN :
 - Simply use the lagged features as inputs for either. MLP easily allow for adding additional input features, and CNN s can exploit local temporal structure by arranging lagged values into structured inputs (e.g., treating them like a small “image”).
- Mixed effects models (which we will briefly touch on in a Bayesian context at the end of the course) are also another strategy.